RESEARCH PREPRINT 17 OCTOBER 2026

The Least Trinomial Multiple of a Primitive Degree-61 Binary Polynomial

Exact exhaustive search and a Lean verification of the witnesses

Sergey Nefedov
Faculty of Mechanics and Mathematics, Lomonosov Moscow State University
Moscow, Russia

Abstract. We determine the least degree of a trinomial multiple of $f(x) = x^{61} + x^{45} + x^{32} + x^2 + 1$ in $\mathbb{F}_2[x]$ within the range $0 < a < b \le 2^{31}$. The least such degree is $b = 848\,144\,186$, attained at $a = 301\,682\,885$. An exhaustive, deterministic search over all 2^{31} positive exponents finds exactly three exponent pairs in the prescribed range. The algorithm uses an exact, 60-bit parity-erased key for each element of a primitive degree-61 finite field, followed by externally partitioned radix sorting and a full recovery pass. We prove that collisions are equivalent to trinomial divisibility and that the enumeration cannot skip any smaller exponent. Independent polynomial remainder calculations verify the three witnesses. A Lean 4 and Mathlib development formalizes the bitset computation and its implication for divisibility in $(\mathbb{Z}/2\mathbb{Z})[X]$; the minimality claim is supported by exhaustive computation, not yet by a Lean kernel proof. Source code and a reproducibility guide accompany this article.

Keywords: finite fields; sparse polynomial multiple; trinomials; exhaustive computation; LFSR; Lean 4; computer-assisted proof.
MSC 2020: 12E20, 12Y05, 68W30, 11T71.

---

1 Problem and contribution

Consider the monic polynomial

$$f(x) = x^{61} + x^{45} + x^{32} + x^2 + 1 \in \mathbb{F}_2[x]. \tag{1}$$

TheoremDB problem P2436 asks for the smallest integer $b \le 2^{31}$ for which there exists a with $0 < a < b$ and

$$f(x) \mid 1 + x^a + x^b. \tag{2}$$

An earlier recorded exact computation excluded $b \le 2^{28}$ [1]. That prefix is not a solution of the requested minimization problem. Our contribution is an explicit least degree and a complete algorithmic exclusion of every smaller degree, together with source code that permits full external reproduction.

Theorem 1 (Least trinomial multiple)**.** *For f given in* (1)*, the smallest integer b satisfying* (2) *with $0 < a < b \le 2^{31}$ is*

$$\boxed{b_0 = 848\,144\,186, \qquad a_0 = 301\,682\,885.}$$

More precisely, the complete set of pairs (a, b) in this range is

$$(301\,682\,885,\ 848\,144\,186),$$
$$(452\,471\,585,\ 1\,293\,170\,928),$$
$$(603\,365\,770,\ 1\,696\,288\,372).$$

The computational part of the proof is specified in Sections 3–4 and released with this preprint. In particular, *the proof of minimality does not follow from verifying the three positive witnesses alone*; it relies on the complete collision enumeration.

2 Exact finite-field model

Set $K = \mathbb{F}_2[x]/(f)$, and write $\alpha = x \bmod f$. The recorded field-model certificate and our standalone verifier check Rabin's irreducibility criterion for the prime degree 61:

$$x^{2^{61}} \equiv x \pmod{f}, \qquad \gcd(f, x^2 + x) = 1. \tag{3}$$

The same verifier evaluates the 59-step Lucas–Lehmer sequence modulo $2^{61} - 1$, obtaining zero. Therefore $2^{61} - 1$ is prime. Since $\alpha \neq 0, 1$, it follows that

$$\operatorname{ord}(\alpha) = 2^{61} - 1. \tag{4}$$

In particular, the residues $\alpha^1, \ldots, \alpha^{2^{31}}$ are pairwise distinct.

Represent an element of K by the integer whose 61 low binary digits are its polynomial coefficients. Let

$$M = 2^{61} - 1, \qquad T = 2^{45} + 2^{32} + 2^2 + 1.$$

For a state $s \in \{0, \ldots, M\}$, multiplication by α is represented exactly by

$$\operatorname{step}(s) = ((2s) \bmod 2^{61}) \ \text{ xor } \begin{cases} T, & \text{if bit 60 of } s \text{ is 1,} \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Here xor is bitwise exclusive OR; it implements addition in characteristic two. Equation (5) follows directly from $x^{61} \equiv x^{45} + x^{32} + x^2 + 1 \pmod{f}$. Starting from state 1, the i-th application produces the exact representation s_i of α^i.

Lemma 2 (Parity-erased collision criterion)**.** *For $1 \le i < j \le 2^{31}$, put $q_i = \lfloor s_i/2 \rfloor$. Then*

$$q_i = q_j \quad \Longleftrightarrow \quad f(x) \mid x^j + x^i + 1. \tag{6}$$

Proof. The quotient by two discards exactly the constant coefficient. Equality of two keys therefore says that s_i and s_j differ at most in bit zero. Their residues are distinct by (4), so the constant coefficients must differ, and s_i xor $s_j = 1$. Conversely, if s_i xor $s_j = 1$, all higher bits agree, hence the keys agree. Because XOR implements addition in K, the former identity is $\alpha^i + \alpha^j + 1 = 0$, equivalent to divisibility by f. □

The use of distinctness is important: without it, a repeated $s_i = s_j$ would also give equal keys but would not give s_i xor $s_j = 1$. Here distinctness is guaranteed by the verified field order; candidate verification additionally checks the exact XOR condition.

3 Complete exact search

By Lemma 2, the task is equivalent to enumerating all duplicate values in the stream

$$q_1, q_2, \ldots, q_B, \qquad B = 2^{31}. \tag{7}$$

Each q_i fits into 60 bits. A key occupies an unsigned 64-bit word; thus the uncompressed raw stream uses $8B = 17\,179\,869\,184$ bytes (16 GiB). To avoid holding it in RAM, the program partitions keys according to the top eight bits

$$p(q) = q \gg 52 \in \{0, 1, \ldots, 255\}.$$

Every equal pair has equal partition index, so partitioning cannot separate a genuine match.

1. **Generate.** Iterate (5) exactly once for every $i = 1, \ldots, B$, including the last endpoint. Append q_i to file $p(q_i)$; count each record and check file sizes.
2. **Partition-local exact sort.** Load each of 256 files in turn, sort full 64-bit words using five stable 11-bit radix passes, and compare adjacent words for literal equality. Since the top eight bits are constant within a partition, the remaining low 52 bits are fully ordered by those passes. Record each duplicate key.
3. **Recover exponents.** Reiterate $s_1, \ldots, s_B$. For each duplicate key recover both exponent labels and check that their full states XOR to one and $0 < a < b \le B$.
4. **Check invariants.** Confirm that 256 partition counts sum to B, the byte counts agree, each collision key has exactly two exponents, and the second pass ends at the same field state.

No hash fingerprint or probabilistic filter is used to accept or reject keys. The key truncation in Lemma 2 is exact for this question, and all sorted comparisons use the entire stored key.

Proposition 3 (Completeness of the search)**.** *If the algorithm above terminates with all 256 partitions successfully processed and the stated invariants satisfied, its output is precisely the set of pairs* (a, b) *with* $0 < a < b \le B$ *and* $f \mid x^b + x^a + 1$.

Proof. Every i in the inclusive range $1 \le i \le B$ is visited by the generation loop, with exactly one key and exactly one partition. For each solution, Lemma 2 gives a pair of equal keys; such keys lie in the same partition. Radix sorting compares exact keys; all duplicates become adjacent, so the pair is detected. Since there are only two 61-bit states with any fixed 60-bit key, and the powers are distinct by (4), no key can occur more than twice. The recovery pass assigns the unique two exponent labels and verifies the defining XOR equality, excluding spurious candidates. Conversely, any emitted pair passes precisely the equivalence in the lemma. □

The finite number of radix passes is independent of B; for fixed word size, generation, radix sorting and recovery require $O(B)$ word operations, although external I/O and memory-bandwidth costs are substantial. Disk space is $8B$ bytes for the keys plus a few smaller metadata files. Processing a partition containing n_p records requires $16n_p$ bytes for the two radix buffers and a small fixed counting array. The code releases each bucket after successful verification. The raw files are regeneration inputs, not retained as part of the distributable source archive.

4 Results and exact arithmetic checks

The complete computation reports the three exponent pairs listed in Theorem 1. The table also lists the independently computed remainders of the monomials modulo f; within each row, the two 61-bit values differ exactly in bit zero.

Table 1: Exact polynomial remainders for the three witnesses.

a	b	$x^a \bmod f$	$x^b \bmod f$
301 682 885	848 144 186	`116ffd86894d162e`	`116ffd86894d162f`
452 471 585	1 293 170 928	`0a3dd96b1419e23d`	`0a3dd96b1419e23c`
603 365 770	1 696 288 372	`0abd85a640cd46f8`	`0abd85a640cd46f9`

The verifier `verify/verify_witness.py` computes these residues by schoolbook carryless polynomial multiplication followed by generic polynomial reduction, an implementation separate from the C++ LFSR transition and radix search. It also compares the two arithmetic methods on a prefix of exponents.

The third row has an algebraic explanation:

$$(x^{848144186} + x^{301682885} + 1)^2 = x^{1696288372} + x^{603365770} + 1 \quad \text{in } \mathbb{F}_2[x]. \tag{8}$$

Because characteristic two eliminates cross terms, it is the square of the first solution. The second row is a distinct solution, not that squaring. The whole search includes even-even pairs, so this square is not accidentally discarded.

4.1 Machine run and consistency checks

The reproducible C++17 implementation completed an inclusive scan of $B = 2\,147\,483\,648$ positive exponents, successfully closing all 256 of 256 partitions and finding exactly three duplicate keys. A second sweep recovered exactly the three exponent pairs of Theorem 1. The process exited with code 0. Summary statistics from its generated log are shown below.

Quantity	Observed value
Enumerated states (inclusive)	2 147 483 648
Partition files processed	256 of 256
Total equal-key pairs	3
Final field state, decimal	263 596 493 844 334 377
Smallest / largest partition sizes	8 379 809 / 8 397 622
Wall-clock time	355.454 s
Peak resident set size (process)	133 468 KiB (130.34 MiB)
Temporary raw key volume	16 GiB

The machine-readable summary and process log are included as `results/full_run_summary.json` and `results/full_search.log`. Their totals and all three witnesses pass the bundled independent Python metadata and remainder checks. The 16 GiB raw intermediate is not shipped with the archive; its exact regeneration is prescribed by the program. This run is a computational certificate whose assurance depends on correct execution of the published source, not a proof checked by the Lean kernel. The main program writes a JSON summary with the inclusive endpoint, all 256 individual bucket counts, the duplicate keys, recovered exponents, the final LFSR state, and elapsed time. A prefix test matches the historical TheoremDB residue after 2,000,000 iterations: 2 021 773 856 510 245 769, with no solutions in that prefix. All three exhibited witnesses pass the independent polynomial remainder checker.

5 Lean proof artifacts and their scope

The source distribution includes two Lean files, supplied in the checked Lean package associated with this project:

- `P2436.lean`, a pure-Lean bitset program with theorems `first_candidate`, `second_candidate`, and `squared_candidate`, using `decide +kernel` rather than `native_decide`.
- `P2436Poly.lean`, a Mathlib development of the bitset-to-polynomial correspondence in the quotient by f. Its soundness theorem lifts a successful Boolean check to ordinary polynomial divisibility, yielding three witness theorems.

The files are pinned to Lean 4.35.0 and Mathlib v4.35.0 in the archive. Their contents were inspected for `sorry`, `admit`, and custom `axiom` declarations; none are present. **The present packaging environment does not contain Lean and could not re-run `lake build`.** Therefore the Lean compilation claims in the supplied archive remain to be verified on an independent Lean installation or by the included GitHub Actions workflow. Neither Lean file proves exhaustive minimality. The division witnesses and exhaustive minimality are two different proof obligations.

6 Reproduction and limitations

The source release contains a standalone C++17 exhaustive search, an independent Python polynomial-arithmetic verifier, the Rabin and Lucas–Lehmer field checks, and regression tests. The essential commands from the repository root are:

```
sh tests/test_search.sh
c++ -O3 -std=c++17 src/exact_search.cpp -o exact_search
./exact_search 2147483648 experiments/full_data
python3 verify/verify_witness.py --summary experiments/full_data/run_summary.json
lake build
```

The full search requires at least 16 GiB of available disk space for raw keys, plus headroom, and enough RAM for the largest two-buffer partition. It is recommended to run it on a native 64-bit platform with adequate free disk and stable power. The raw 16 GiB stream is not included with the GitHub source ZIP; it can be reconstructed exactly. The primary scan was executed in this study, but an independent *second full search program* is not included in this release. Source audit, independent witness verification and externally reproducing the full exhaustive search remain important for high-assurance use of the minimality conclusion.

Prior algorithms for low-weight polynomial multiples, including Zech-logarithm and discrete-logarithm methods, are studied by Didier–Laigle-Chapuy [2] and Peterlongo–Sala–Tinnirello [3]; sparse multiples in a broader algebraic setting are studied in [4]. We do not claim that radix sorting or out-of-core collision search is a new algorithmic paradigm. The result is the computed least degree for this particular, previously documented instance. A literature search identified no earlier public value for this specific minimal pair, but this is not an exhaustive priority audit.

Code and data availability

The complete source, exact enumeration record, and reproduction instructions accompany this preprint as a GitHub-ready archive. The raw temporary 16 GiB key stream is not included; deterministic regeneration is described and implemented.

References

[1] TheoremDB, *P2436: Least trinomial multiple of a primitive degree-61 polynomial*, research record, reviewed 24 July 2026. https://theoremdb.org/statements/P2436/.

[2] F. Didier and Y. Laigle-Chapuy, *Finding low-weight polynomial multiples using discrete logarithm*, IEEE International Symposium on Information Theory, 2007, pp. 1036–1040. https://arxiv.org/abs/cs/0701069.

[3] P. Peterlongo, M. Sala, and C. Tinnirello, *A discrete logarithm-based approach to compute low-weight multiples of binary polynomials*, Finite Fields and Their Applications **38** (2016), 57–71. https://doi.org/10.1016/j.ffa.2015.12.005.

[4] M. Giesbrecht, D. Roche, and H. Tilak, *Computing sparse multiples of polynomials*, arXiv:1009.3214. https://arxiv.org/abs/1009.3214.

Appendix A. Reproducibility audit

Observed environment. The fresh exhaustive search was compiled with GCC 14.2.0 on Linux x86-64 (Intel Xeon Platinum 8573C). A separate Python interpreter was used for the reference polynomial remainder computations and the field-property checks. The program reported an elapsed time of 355.454 seconds, a peak RSS of 133 468 KiB, and successful termination of the complete scan.

Recorded integrity checks. The SHA-256 digests of the completed machine-readable summary and process log are, respectively:

```
9531cf943ab223532be089c498c9f54ee55187ba8b7d8f50039235df239f18bc
7c8ee6ced0def78db8fa84c8a180197808eba549dfc378e9eedb86960dca562d
```

These digests protect file integrity only. They are not substitutes for the correctness proof in Proposition 3, auditing the C++ enumeration, or an independent full rerun.

Concrete replay. A minimal independent audit uses:

```
python3 verify/verify_field.py
python3 verify/verify_witness.py
python3 verify/audit_summary.py results/full_run_summary.json
sh tests/test_search.sh
```

The complete scan command in Section 6 additionally regenerates all 256 partitions and verifies their exact keys before removing them. The partial-reference Python dictionary implementation may be used for smaller bounds, as a second method to check the indexing conventions.

Limits of formal certification. The supplied Lean files are included unchanged from their checked-source package. A source inspection found no proof holes, but a new Lean kernel replay requires an external Lean environment. Even after all Lean witness theorems compile, the universal minimality statement is still established by the exhaustive C++ enumeration, not by a Lean theorem.